

Classical Linear Search vs. Grover’s Quantum Search

A Real Qiskit-Simulated Benchmark

1 Overview

This document explains `linear_vs_grover_search.py`, a script that benchmarks classical linear search against Grover’s quantum search algorithm. Unlike a purely theoretical comparison, the quantum side is *actually built and executed* as a real circuit on Qiskit’s `AerSimulator`, for every search-space size that fits inside the configured qubit and RAM budget. The classical side counts real comparisons made while scanning a list.

The script answers a single practical question: as the search space N grows from 10 to millions of items, how many queries does each approach actually need, and how does that translate into real simulator qubits, RAM, and wall-clock time?

2 Algorithms Implemented

2.1 Classical Linear Search

`classical_linear_search(items, target)` scans a list element by element, incrementing a comparison counter until the target is found. This gives:

- **Worst case:** N queries (target is last, or absent).
- **Average case:** $\frac{N + 1}{2}$ queries.

No shortcuts are taken — the counter reflects genuine iteration, so the numbers in the results table are not just $O(N)$ asymptotics but the exact comparison counts for each N .

2.2 Grover’s Quantum Search

`build_grover_circuit(n_qubits, marked_state, max_iterations)` constructs a real Grover circuit:

1. **Superposition:** a Hadamard gate on every data qubit prepares an equal superposition over all $N = 2^n$ basis states.
2. **Oracle:** phase-flips the single marked basis state. Qubits that should be 0 in the target are temporarily flipped with X gates so the target maps to $|1 \dots 1\rangle$, a multi-controlled-X (via an H –MCX– H sandwich) applies the phase flip, and the X gates are undone.
3. **Diffuser:** inversion about the mean, built the same way (H, X, controlled phase flip, X, H) so amplitude is reflected toward the marked state.

4. **Iteration:** oracle + diffuser form one `GroverStep`. The optimal number of repetitions is the well-known result

$$r^* = \left\lfloor \frac{\pi}{4} \sqrt{N} \right\rfloor,$$

which is computed exactly as `true_iterations`. Because this can reach into the thousands for large N , a separate `max_iterations` cap (`sim_iterations`) limits how many repetitions are actually *simulated*, while the theoretical query-count columns in the results table always use the true, uncapped value.

For 1 qubit, `MCXGate` has no valid control count, so a dedicated single-qubit oracle/diffuser branch is used instead of the general multi-controlled construction.

2.3 Why `.repeat()` Instead of a Python Loop

An earlier version of the script called `circuit.compose()` once per Grover iteration. At realistic iteration counts (thousands, for $N10^6$) this froze circuit *construction* — not simulation — because each `compose()` call reprocesses an ever-growing circuit object. The fix used here builds the oracle+diffuser pair once as a gate (`qc_step.to_gate(label="GroverStep")`) and stamps it `sim_iterations` times with `gate.repeat(n)`. This produces a single compact custom instruction instead of thousands of discrete circuit-composition calls.

That compactness has one consequence: `AerSimulator` cannot execute an opaque custom instruction directly (this is the source of the `AerError: unknown instruction: GroverStep` error encountered during development). The fix is to `transpile(circuit, sim)` before calling `sim.run(...)`, which decomposes `GroverStep` — and the ancilla/multi-controlled gates inside it — down to the simulator’s native basis gates.

3 RAM and Qubit Budget

A statevector simulator stores one complex amplitude per basis state:

$$\text{memory (bytes)} = 2^{n_{\text{qubits}}} \times 16,$$

using 16 bytes per `complex128` amplitude. The script exposes this as `statevector_gb(n_qubits)` and gates whether a given case is actually simulated via two independent limits:

- `RAM_BUDGET_BYTES` — the memory ceiling (configured at 8 GiB in this run).
- `SAFE_SIM_QUBITS` — a separate qubit-count ceiling (24 in this run), independent of raw memory, since circuit-construction and gate-simulation *time* grow with qubit count and iteration count even when the statevector itself is small.

A case is only marked **YES** (`real Aer run`) when it satisfies both limits (`can_actually_simulate_in_4GB` in the code, named for the original 4 GB constraint). Everything else would fall back to `theoretical_grover()`, which computes the closed-form Grover success probability

$$P_{\text{success}} = \sin^2((2r + 1)\theta), \quad \theta = \arcsin\left(\frac{1}{\sqrt{N}}\right)$$

without building or running a circuit at all.

4 Results

The table below is the script’s actual output for $N \in \{10, 1000, 100000, 500000, 1,000,000, 2,000,000\}$. Every row in this run satisfied the RAM/qubit budget, so every row is a genuine `AerSimulator` execution — none fell back to the theoretical-only branch.

N	Qubits	Classical (worst)	Classical (avg)	Quantum (theory)	Speedup	RAM (GB)	Simulated?
10	4	10	5.5	2	5.00	≈ 0	Yes (Qiskit-Aer)
1,000	10	1,000	500.5	25	40.00	1.5×10^{-5}	Yes (Qiskit-Aer)
100,000	17	100,000	50,000.5	248	403.23	0.00195	Yes (Qiskit-Aer)
500,000	19	500,000	250,000.5	555	900.90	0.00781	Yes (Qiskit-Aer)
1,000,000	20	1,000,000	500,000.5	785	1273.89	0.01563	Yes (Qiskit-Aer)
2,000,000	21	2,000,000	1,000,000.5	1,111	1800.18	0.03125	Yes (Qiskit-Aer)

Table 1: Classical vs. theoretical-quantum query counts. Speedup is worst-case classical queries divided by theoretical Grover queries.

N	Iterations run	Measured P_{success}	Runtime (s)	Statevector (GB)
10	3/3	0.9629	0.085	≈ 0
1,000	25/25	0.9990	0.091	0.000015
100,000	284/284	≈ 1	5.544	0.001953
500,000	569/569	≈ 1	37.799	0.007812
1,000,000	804/804	≈ 1	112.640	0.015625
2,000,000	1137/1137	≈ 1	334.105	0.031250

Table 2: Actual `AerSimulator` execution results: iterations run out of the theoretical optimum, measured success probability over 2048 shots, wall-clock simulation time, and statevector memory footprint.

5 Analysis

- **Query speedup grows with $\sqrt{N}$, as expected.** The ratio of classical worst-case queries to Grover’s theoretical queries climbs from $5\times$ at $N = 10$ to roughly $1800\times$ at $N = 2,000,000$, consistent with the $O(N)$ vs. $O(\sqrt{N})$ separation.
- **Statevector memory stays negligible throughout this range.** Even at 21 qubits the statevector is only 0.031 GiB. Memory is not the bottleneck in any of these runs — it only becomes a constraint much closer to the 29-qubit cap mentioned in the script’s header comments ($2^{29} \times 16$ bytes ≈ 8.6 GiB).
- **Runtime, not memory, is the real constraint at scale.** Wall-clock time grows sharply with iteration count: from 0.085 s at 3 iterations to 334 s at 1137 iterations. This is roughly linear in iteration count times the per-iteration gate-simulation cost, which itself grows with qubit count — explaining why $N = 2,000,000$ already takes over five and a half minutes despite using barely 0.03 GiB of memory.
- **All measured success probabilities match theory closely.** Every case with a nontrivial iteration count converges to (or very near) unit success probability, matching the closed-form

$P_{\text{success}} = \sin^2((2r + 1)\theta)$ formula — confirming the oracle/diffuser construction, the ancilla-mode multi-controlled gates, and the `transpile()` step are all behaving correctly.

- **Practical implication.** The bottleneck for pushing this benchmark toward the full 29-qubit / hundreds-of-millions-of-items regime is simulation *time* (iteration count $\times$ per-iteration gate cost), not RAM. Reaching those sizes on a statevector simulator would require either capping simulated iterations far below the theoretical optimum (as `max_iterations` already allows), or abandoning full circuit simulation in favor of the theoretical formulas via `theoretical_grover()`.

6 Configuration Reference

```
MAX_QUBITS = 29 # requested search-space cap (2^29 items)
SAFE_SIM_QUBITS = 24 # qubit ceiling for actually running a circuit
RAM_BUDGET_BYTES = 8 * 1024 ** 3 # memory ceiling for the RAM/qubit check in bytes
QUERY_SIZES = [10, 1_000, 100_000, 500_000, 1_000_000, 2_000_000]
```

Raise `SAFE_SIM_QUBITS` or extend `QUERY_SIZES` to push further toward the 29-qubit cap; expect runtime, not memory, to be the limiting factor based on the scaling observed above.